



# **INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO.**



- **Alumno:**

**Rodriguez Rosales Diego Guillermo.**

- **Num. Control:**

**#222310306.**

- **Docente:**

**Jesus Salas Marin.**

- **Materia:**

**Desarrollo de Aplicaciones para Dispositivos Móviles.**

- **Clave:**

**4Y8A.**

- **Actividad:**

**Pra 2.4.7 Creando y Manipulando Clases en Dart.**

**23-Marzo-2026.**





### **Paso 1. Creación de la Clase Caja.**

1. Crea una clase Caja con los atributos anchura, altura y longitud.
2. Implementa un método para calcular volumen y otro para calcular el área total de la superficie.

```
1 class Caja {  
2     double ancho = 0.0;  
3     double alto = 0.0;  
4     double largo = 0.0;  
5  
6     //constructor  
7     Caja(this.ancho, this.alto, this.largo);  
8  
9     //metodo para calcular volumen  
10    double calcularVolumen() {  
11        return ancho * alto * largo;  
12    }  
13    //metodo para calcular areaTotal  
14    double calcularAreaTotal() {  
15        return 2 * (ancho * alto + ancho * largo + alto * largo);  
16    }  
17 }
```

- Archivo de prueba (prueba1.dart).

```
1 import 'Caja1.dart';  
2  
3 void main(){  
4     Caja miCaja = Caja(2.5, 5.0, 6.0);  
5     print("Área: ${miCaja.calcularAreaTotal()}");  
6     print("Volumen: ${miCaja.calcularVolumen()}");  
7 }
```

- ☐ Ejecutamos el archivo prueba1.dart y muestra lo siguiente:

```
PS C:\Users\dgrn2\Downloads> dart run prueba1.dart  
Área: 115.0  
Volumen: 75.0
```

### **Paso 2. Creando una Clase Caja con propiedades privadas.**

1. Haz privadas las variables de instancia.
2. Implementa métodos de acceso (get) para obtener los valores sin permitir modificaciones.



### 3. Prueba el código con un programa de prueba.

```
1 class Caja {
2   double _ancho;
3   double _alto;
4   double _largo;
5
6   // Constructor
7   Caja(this._ancho, this._alto, this._largo);
8
9   // Getters
10  double get ancho => _ancho;
11  double get alto => _alto;
12  double get largo => _largo;
13
14  // Método para calcular volumen
15  double calcularVolumen() {
16    return _ancho * _alto * _largo;
17  }
18
19  // Método para calcular área total
20  double calcularAreaTotal() {
21    return 2 * (_ancho * _alto +
22              _ancho * _largo +
23              _alto * _largo);
24  }
25 }
```

- Archivo de prueba (prueba2.dart).

```
1 import 'Caja2.dart';
2
3 void main(){
4   Caja miCaja = Caja(2.5, 5.0, 6.0);
5   print("Área: ${miCaja.calcularAreaTotal()}");
6   print("Volumen: ${miCaja.calcularVolumen()}");
7
8   // Usando acceso a propiedades privadas por medio de getters
9   print("Dimensiones -> Ancho: ${miCaja.ancho}, Alto: ${miCaja.alto}, Largo: ${miCaja.largo}");
10
11 }
```

☐ Ejecutamos el archivo prueba2.dart y muestra lo siguiente:

```
PS C:\Users\dgr2\Downloads> dart run prueba2.dart
Área: 115.0
Volumen: 75.0
Dimensiones -> Ancho: 2.5, Alto: 5.0, Largo: 6.0
```



### ***Paso 3. Métodos Privados para Calcular Área.***

1. Reescribe el método "areaTotal()" para usar métodos privados que calculen el área de cada cara de la caja.

```
1 class Caja {
2     double _ancho;
3     double _alto;
4     double _largo;
5
6     //constructor
7     Caja(this._ancho, this._alto, this._largo);
8
9     // Métodos de acceso (Getters)...
10    double get ancho => _ancho;
11    double get alto => _alto;
12    double get largo => _largo;
13
14    // Métodos privados para calcular áreas parciales
15    double _areaFrontal() => _ancho * _alto;
16    double _areaSuperior() => _ancho * _largo;
17    double _areaLateral() => _alto * _largo;
18
19    //metodo para calcular areaTotal
20    double areaTotal() => 2 * (_areaFrontal() + _areaSuperior() + _areaLateral());
21
22    //metodo para calcular volumen
23    double volumen() => _ancho * _alto * _largo;
24 }
```

- Archivo de prueba (prueba3.dart).

```
1 class Caja {
2     double _ancho;
3     double _alto;
4     double _largo;
5
6     //constructor
7     Caja(this._ancho, this._alto, this._largo);
8
9     // Métodos de acceso (Getters)...
10    double get ancho => _ancho;
11    double get alto => _alto;
12    double get largo => _largo;
13
14    // Métodos privados para calcular áreas parciales
15    double _areaFrontal() => _ancho * _alto;
16    double _areaSuperior() => _ancho * _largo;
17    double _areaLateral() => _alto * _largo;
18
19    //metodo para calcular areaTotal
20    double areaTotal() => 2 * (_areaFrontal() + _areaSuperior() + _areaLateral());
21
22    //metodo para calcular volumen
23    double volumen() => _ancho * _alto * _largo;
24 }
```

- ☐ Ejecutamos el archivo prueba3.dart y muestra lo siguiente:



```
PS C:\Users\dgrr2\Downloads> dart run prueba3.dart
Área: 115.0
Volumen: 75.0
```

#### ***Paso 4. Constructor de Copia y Métodos de Acceso.***

1. Agrega un constructor de copia para crear una nueva caja con dimensiones idénticas a otra.
2. Implementa métodos de acceso para las dimensiones.

```
1 class Caja {
2   double _ancho;
3   double _alto;
4   double _largo;
5
6   // Constructor normal
7   Caja(this._ancho, this._alto, this._largo);
8
9   // Constructor de copia
10  Caja.clonarDe(Caja otra)
11    : _ancho = otra._ancho,
12      _alto = otra._alto,
13      _largo = otra._largo;
14
15  // Métodos de acceso (Getters)
16  double get ancho => _ancho;
17  double get alto => _alto;
18  double get largo => _largo;
19
20  // Métodos privados para calcular áreas parciales
21  double _areaFrontal() => _ancho * _alto;
22  double _areaSuperior() => _ancho * _largo;
23  double _areaLateral() => _alto * _largo;
24
25  // Método para calcular areaTotal
26  double areaTotal() =>
27    2 * (_areaFrontal() + _areaSuperior() + _areaLateral());
28
29  // Método para calcular volumen
30  double volumen() => _ancho * _alto * _largo;
31 }
```

- Archivo de prueba (prueba4.dart).

```
1 import 'Caja4.dart';
2
3 void main() {
4   Caja cajaOriginal = Caja(2.5, 5.0, 6.0);
5   Caja cajaCopia = Caja.clonarDe(cajaOriginal);
6
7   print("Caja Copia -> Ancho: ${cajaCopia.ancho}, Alto: ${cajaCopia.alto}, Largo:${cajaCopia.largo}");
8
9 }
```

- ☐ Ejecutamos el archivo prueba4.dart y muestra lo siguiente:

```
PS C:\Users\dgrr2\Downloads> dart run prueba4.dart
Caja Copia -> Ancho: 2.5, Alto: 5.0, Largo:6.0
```



## Paso 5. Creando una Caja Más Grande.

1. Implementa el método “biggerBox()” para generar una caja un 25% más grande.
2. Implementa el método “smallerBox()” para generar una caja un 25% más pequeña.

```
1 class Caja {
2     double _ancho;
3     double _alto;
4     double _largo;
5
6     // Constructor normal
7     Caja(this._ancho, this._alto, this._largo);
8
9     // Constructor de copia
10    Caja.clonarDe(Caja otra)
11        : _ancho = otra._ancho,
12          _alto = otra._alto,
13          _largo = otra._largo;
14
15    // Métodos de acceso (Getters)
16    double get ancho => _ancho;
17    double get alto => _alto;
18    double get largo => _largo;
19
20    // Métodos privados para calcular áreas parciales
21    double _areaFrontal() => _ancho * _alto;
22    double _areaSuperior() => _ancho * _largo;
23    double _areaLateral() => _alto * _largo;
24
25    // Método para calcular área total
26    double areaTotal() =>
27        2 * (_areaFrontal() + _areaSuperior() + _areaLateral());
28
29    // Método para calcular volumen
30    double volumen() => _ancho * _alto * _largo;
31
32    // Método para crear una caja 25% más grande
33    Caja biggerBox() {
34        return Caja(_ancho * 1.25, _alto * 1.25, _largo * 1.25);
35    }
36
37    // Método para crear una caja 25% más pequeña
38    Caja smallerBox() {
39        return Caja(_ancho * 0.75, _alto * 0.75, _largo * 0.75);
40    }
41 }
```

- Archivo de prueba (prueba5.dart).

```
1 import 'Caja5.dart';
2
3 void main() {
4     Caja cajaOriginal = Caja(2.5, 5.0, 6.0);
5     Caja cajaGrande = cajaOriginal.biggerBox();
6     Caja cajaPequena = cajaOriginal.smallerBox();
7
8     print('Caja más grande -> Ancho: ${cajaGrande.ancho}, Alto: ${cajaGrande.alto}, Largo: ${cajaGrande.largo}');
9     print('Caja más pequeña -> Ancho: ${cajaPequena.ancho}, Alto: ${cajaPequena.alto}, Largo: ${cajaPequena.largo}');
10 }
11
```

- ☐ Ejecutamos el archivo prueba5.dart y muestra lo siguiente:

```
PS C:\Users\dgr2\Downloads> dart run prueba5.dart
Caja más grande -> Ancho: 3.125, Alto: 6.25, Largo: 7.5
Caja más pequeña -> Ancho: 1.875, Alto: 3.75, Largo: 4.5
```



## Paso 6. Evaluar si una caja cabe dentro de otra.

1. Implementa el método “nests()” para verificar si una caja cabe dentro de otra.

```
1  class Caja {
2
3      double _ancho;
4      double _alto;
5      double _largo;
6
7      // Constructor normal
8      Caja(this._ancho, this._alto, this._largo);
9
10     // Constructor de copia
11     Caja.clonarDe(Caja otra)
12         : _ancho = otra._ancho,
13           _alto = otra._alto,
14           _largo = otra._largo;
15
16     // Métodos de acceso (Getters)
17     double get ancho => _ancho;
18     double get alto => _alto;
19     double get largo => _largo;
20
21     // Métodos privados para calcular áreas parciales
22     double _areaFrontal() => _ancho * _alto;
23     double _areaSuperior() => _ancho * _largo;
24     double _areaLateral() => _alto * _largo;
25
26     // Método para calcular área total
27     double areaTotal() =>
28         2 * (_areaFrontal() + _areaSuperior() + _areaLateral());
29
30     // Método para calcular volumen
31     double volumen() => _ancho * _alto * _largo;
32
33     // Método para crear una caja 25% más grande
34     Caja biggerBox() {
35         return Caja(_ancho * 1.25, _alto * 1.25, _largo * 1.25);
36     }
37
38     // Método para crear una caja 25% más pequeña
39     Caja smallerBox() {
40         return Caja(_ancho * 0.75, _alto * 0.75, _largo * 0.75);
41     }
42
43     // Método para verificar si una caja cabe dentro de otra
44     bool nests(Caja otra) {
45         return _ancho < otra.ancho &&
46             _alto < otra.alto &&
47             _largo < otra.largo;
48     }
49 }
```



- Archivo de prueba (prueba6.dart).

```
1 import 'Caja6.dart';
2
3 void main() {
4     Caja caja1 = Caja(2.5, 5.0, 6.0);
5     Caja caja2 = Caja(3.0, 6.0, 7.0);
6
7     print("¿Caja1 cabe dentro de Caja2? ${caja1.nests(caja2)}");
8 }
```

- ☐ Ejecutamos el archivo prueba6.dart y muestra lo siguiente:

```
PS C:\Users\dgrr2\Downloads> dart run prueba6.dart
¿Caja1 cabe dentro de Caja2? true
```



## **Conclusión.**

En esta práctica aprendí a crear y manipular clases en Dart aplicando los principios fundamentales de la Programación Orientada a Objetos. A lo largo de los ejercicios trabajé con encapsulación utilizando variables privadas y métodos getters para controlar el acceso a los datos. También implementé constructores normales y de copia, lo que me permitió comprender cómo crear nuevas instancias de una clase a partir de objetos existentes.

Además, desarrollé métodos privados para organizar mejor el cálculo de áreas, así como métodos que retornan nuevos objetos, como las funciones para generar cajas más grandes o más pequeñas. Finalmente, implementé un método para comparar objetos y determinar si una caja puede contener a otra.

Esta práctica me ayudó a reforzar conceptos importantes como clases, métodos, constructores, encapsulación y manipulación de objetos en Dart, los cuales son fundamentales para el desarrollo de aplicaciones móviles.